



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A table - driven agent wants one table entry for every possible percepts sequence. With $|P|$ possible percepts and an agent lifetime of T steps, that is $|P|^T$ entries. So as the lifetime increases, the table grows exponentially. Chess has about 10^{40} legal board states and about 10^{120} possible game sequences (the Shannon number), so no storage could hold the table and nobody could write it. That is why we write agent programs instead.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Condition-Action Rules in SimpleReflexAgent

The rules are the three IF-THEN lines in `sense_and_act`:

1. `if percent.get("food_here"):`
 `return "Suck"`
2. `if percent.get("wall_ahead"):`
 `return "Left"`
3. `return "Right"` as the default

Each one maps the current percepts straight to an action. The class has no `_int_` and no stored data, so it has zero memory, which is exactly what the lecture describes for reflex agents.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The percept has only local booleans, so the agent never sees its position or anything beyond the next cell, and it keeps no percept history. In my run the agent ate the single food on the bottom row (score 20) and run to the right edge. There `wall_ahead` gave Left, so it moved back one cell. There was no wall, so the default Right took it back, and it repeated this two-cell cycle until the 150-step limit. If left 14 of 15 food uneaten, because with the same percept it always chooses the same action and can't tell it has been there before.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

ModelBasedAgent keeps an internal state of (x, y, facing, visited, blocked, toxic), and update_state runs before the rules on every step.

1. Sensor Model-> How the agent's action affect to the world.

*It applies the last action to its remembered position and facing, unless the move bumped, and it counts visited cells.

2. Transition Model -> How the world involves by itself.

*Walls and toxic cells don't move, so what it has seen stays in memory as blocked and toxic.

The rules then use that memory to skip blocked directions, prefer the least-visited neighbour and avoid toxic cells. In my run this let it explore the grid and eat 13 of 15 food for a score of 185, while the simple agent scored 20. The remaining loss came from bumps and trap steps, because it tracks its position by counting moves and learns walls only after seeing them.



Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING